

First-Timer Registration Flow

A patient who books over WhatsApp is not yet a patient. Today they become one anyway — spending a medical record number and creating a 25-year record for someone who may never walk in. This is where the nullable columns come from, and how to remove them.

STATUS	DATE	PHASE	EFFORT	ALREADY BUILT
Proposal — for review	30 August 2026	P17-T01 ... P17-T06	≈37 pts	PCS-T08 arrival worklist & merge

Why this document exists

The PRD needed a patient's date of birth to password-protect a delivered document, and asked what happens when it is null. The answer — *"the front desk captures it at registration"* — is right for someone standing at the counter, and does not describe the case the null exists for.

The product owner named that case precisely:

NOTE

"The reason why the patient data is nullable is for the new patient that wants to visit the clinic via WA or Telegram. The chatbot restricts any kind of data except name and phone number."

So the nullability is not sloppiness. It is a deliberate accommodation, recorded in the schema itself:

NOTE

Nullable since `PCS-T07`. Every record a human creates still supplies it — `createPatientSchema` keeps it required — but a chat-created draft genuinely does not have one, and the alternatives were both worse than a null: refusing to book, or writing a placeholder date into a record that PMK 24/2022 says is kept for 25 years.

That reasoning holds. What it reveals is a different problem, and the product owner's two suggestions are the fix.

What already exists — read this before building anything

PCS-T08 shipped the arrival flow. Roughly seventy per cent of the "new first-timer flow" is already in the product:

PIECE	STATE
Arrival worklist — today's channel bookings whose record nobody has filled in	Ships. <code>ChannelArrivalService.listArrivals</code> , defaulted to the clinic's today in <code>CLINIC_TIMEZONE</code> , surfaced at <code>channel-arrivals-panel.tsx</code>
Merge — the person standing there turns out to already be a patient	Ships. <code>channel-arrival-merge-dialog.tsx</code> ; the service's docstring names the exact cases: "the phone number they typed was a family member's, or verification never happened"
Missing-field visibility	Ships. <code>channel-arrival-missing-fields.tsx</code>
Completing the record	Ships, deliberately elsewhere. Through the ordinary patient-edit route, because "a second write path for the same columns is a second place for the encryption rules to drift"
Verified phone number	Ships. <code>ChannelPatientLink</code> + <code>Channel0tpChallenge</code>

The strategy sentence this implements — *"arrival completes the record"* — is already the right sentence. **This document does not propose a new flow. It proposes moving where the record lives before arrival**, which is the one thing that stops the existing flow from being able to require the fields.

The cost nobody has priced

A chat booking today creates a real `PatientProfile`. That has three consequences that follow from the repo's own rules.

It spends an MRN. `PatientProfile.mrn` is `@unique` and allocated from `MrnCounter` by an atomic `INSERT ... ON CONFLICT ... RETURNING` inside the create transaction. A medical record number is issued to anyone who types a name and a phone number into WhatsApp. A stranger who books and never arrives has permanently consumed one, and the sequence never reuses it.

It creates a medical record for someone who was never a patient. PMK 24/2022 retention applies to the row, not to whether the person attended. The clinic is holding, for twenty-five years, a record of a name and a phone number belonging to somebody who may have been a wrong number.

It is why the columns are nullable. Date of birth, sex and address are optional in the database *only* so that this row can exist. Every downstream feature then has to handle a null — the PRD hit it at `FR-E4-07`, and it will keep surfacing.

None of this is an argument that PCS-T07 was wrong. Given a schema where the only place to put a booking was PatientProfile, a null beat a placeholder. The fix is to give it somewhere else to go.

Proposal

A prospective patient is not a patient

Introduce a staging record for someone who has asked for an appointment but has never attended.

```
/// Someone who asked for an appointment through a messaging channel and has
/// not yet arrived. Deliberately NOT a PatientProfile: no MRN is spent, no
/// medical record exists, and the 25-year RME retention floor does not apply
/// to a person who was never a patient.
///
/// The chatbot can only collect a name and a phone number (PCS-T07, strategy
/// $5.1), and that is exactly what this table requires. Everything a clinical
/// record needs is collected at the counter, where a human is looking at an ID.
model ProspectivePatient {
    id                String                @id @default(uuid()) @db.Uuid
    fullName          String                @map("full_name")
    /// Normalised the same way `ChannelPatientLink.phoneNumber` is, so matching
    /// against the registry at arrival is an exact comparison.
    phoneNumber       String                @map("phone_number")
    channel           ChannelKind
    externalChatId    String?               @map("external_chat_id")
    status            ProspectivePatientStatus @default(AWAITING_ARRIVAL)
    /// Set when the record becomes, or is matched to, a real patient. The link
    /// is kept rather than the row deleted: it is how a duplicate booking from
    /// the same number resolves to the same person next time.
    patientId         String?               @map("patient_id") @db.Uuid
    convertedAt       DateTime?             @map("converted_at")
    convertedById     String?               @map("converted_by_id") @db.Uuid
    expiresAt        DateTime              @map("expires_at")
    createdAt         DateTime              @default(now()) @map("created_at")
    updatedAt         DateTime              @updatedAt @map("updated_at")

    patient           PatientProfile? @relation(fields: [patientId], references: [id], onDelete: SetNull)
    appointments      Appointment[]

    @@index([status, expiresAt])
    @@index([phoneNumber])
    @@map("prospective_patients")
}

enum ProspectivePatientStatus {
    AWAITING_ARRIVAL
    /// Became a new PatientProfile at the counter.
    CONVERTED
    /// Matched to a PatientProfile that already existed – the merge case
    /// `PCS-T08` already handles.
    LINKED
    /// Never arrived. Expired and purged; no MRN was ever spent.
```

What the appointment points at

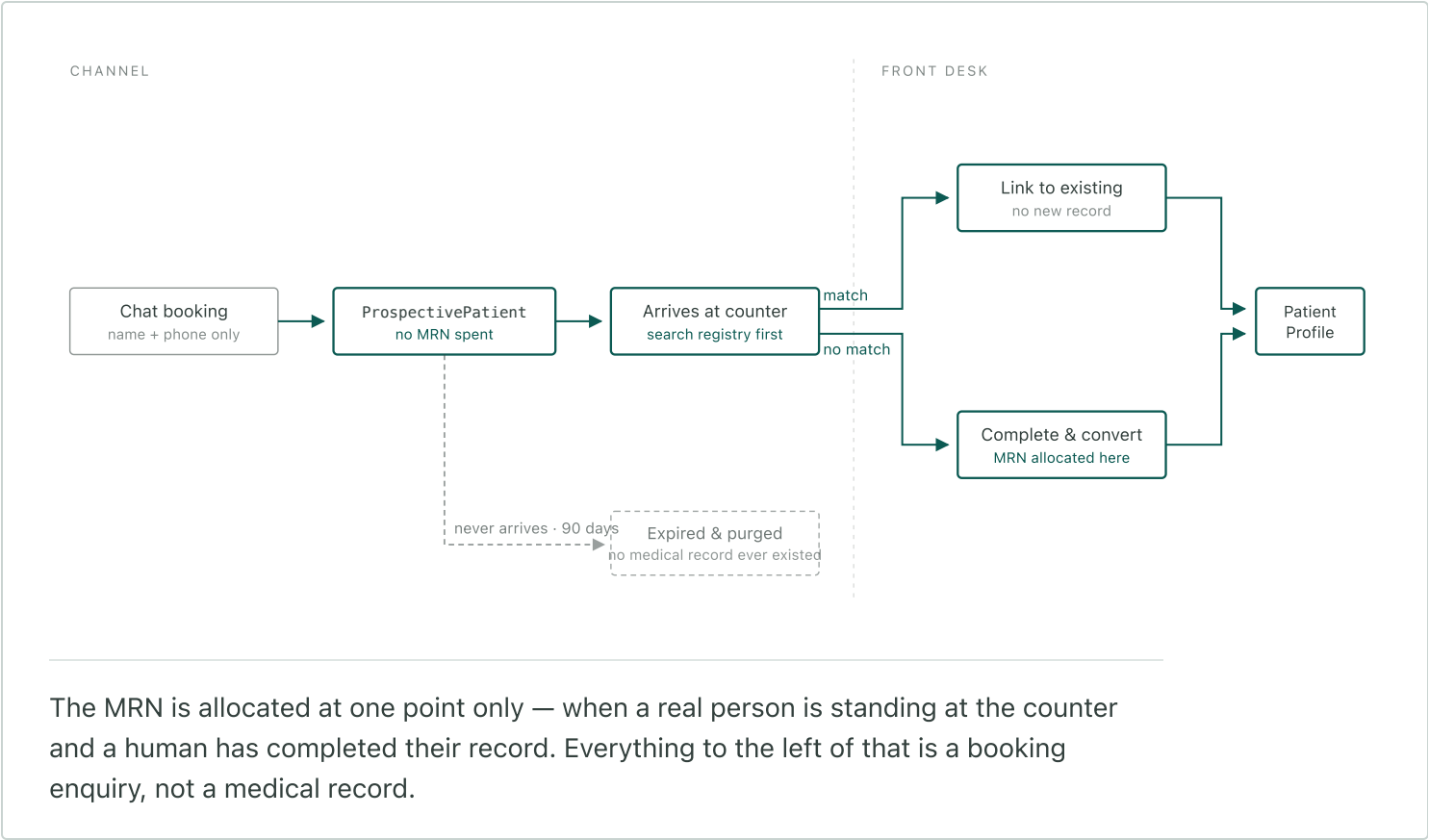
`Appointment.patientId` is non-null today, which is precisely why a chat booking had to invent a patient. It becomes a pair of nullable foreign keys with a CHECK that exactly one is set — the `Invoice.encounterId / admissionId` pattern this repo already uses, whose schema comment reads "exactly one of `encounterId` and `admissionId` is set, enforced by a CHECK in the migration — an invoice always says which episode of care it bills, and never two."

```
model Appointment {
  /// Exactly one of these is set, enforced by a CHECK. A booking either names
  /// a patient the clinic has, or a person who has asked to become one.
  patientId          String? @map("patient_id") @db.Uuid
  prospectivePatientId String? @map("prospective_patient_id") @db.Uuid
}
```

Conversion **repoints the appointment** in the same transaction that creates or matches the patient. Nothing else about scheduling changes.

The flow

1. **Chat booking.** The bot collects a name and a phone number — all it is allowed to collect — and creates a `ProspectivePatient` plus an `Appointment` pointing at it. **No MRN is allocated.**
2. **Confirmation.** Unchanged: the existing booking-confirmation reply, with the existing arrival instruction.
3. **Arrival.** The person appears at the counter. They are already on the arrival worklist `PCS-T08` built.
4. **Search first.** The front desk searches the registry by name, phone and NIK. This step is not optional — it is what stops a returning patient who booked from a new phone becoming a second record.
5. **Branch: - Match found → link.** The appointment repoints to the existing `PatientProfile`; the prospective record is marked `LINKED`. This is the merge `PCS-T08` already implements. - **No match → convert.** The front desk completes the required demographics from an ID document. `PatientProfile` is created, **the MRN is allocated here**, and the appointment repoints. The prospective record is marked `CONVERTED`.
6. **Registration proceeds** exactly as it does for a walk-in today.
7. **Never arrives.** The prospective record expires and is purged. No MRN was spent, and no medical record was ever created.



Making the columns required

This is the product owner's first suggestion, and it is the payoff. It cannot be one migration — the existing rows have to go somewhere first.

Three releases, in order:

#	RELEASE	WHAT HAPPENS
1	Add	<code>ProspectivePatient</code> , the nullable <code>Appointment.prospectivePatientId</code> , the CHECK. The chat path starts writing prospective records. Nothing existing changes.
2	Drain	Backfill existing <code>CHANNEL_BOOKING</code> drafts. A draft with no clinical activity — no encounter, no registration, no prescription, no invoice — becomes a <code>ProspectivePatient</code> and its <code>PatientProfile</code> is removed. A draft with activity means that person did attend, so it stays and the front desk completes it. Report the count of each before running.
3	Tighten	Only once release 2 reports zero incomplete profiles: <code>dateOfBirth</code> , <code>sex</code> and <code>address</code> become <code>NOT NULL</code> .

Release 3 must not be attempted before release 2 reports clean, and the migration should refuse to run if any row would violate the new constraint — a failed deploy is better than a placeholder date written into a record kept for twenty-five years.

What becomes required, and what stays optional

Required after release 3 — the fields `createPatientSchema` **already** demands of every human-created record, so this only aligns the database with the API:

fullName , dateOfBirth , sex , phoneNumber , address

Staying optional, each for a stated reason:

FIELD	WHY IT STAYS OPTIONAL
nik	The schema already says it: "newborns have no NIK for weeks, foreign nationals carry a passport or KITAS, and an unidentified emergency arrival needs a record immediately"
bpjsNumber	Not every patient is a BPJS member
IHS / SATUSEHAT identifiers	Assigned by an external system, often after registration
Doctor assignment	A relationship, not an attribute of the person
placeOfBirth , email , bloodType , rhesusFactor , maritalStatus , occupation , religion	Demographics PMK 24/2022 expects but that a counter cannot always obtain on the first visit
emergencyContactName / Phone , guardianName / Relation	Situational — a guardian is required in practice for a minor and meaningless for an adult

Retention for a record that never converts

A prospective patient is **not** clinical data, so the 25-year RME floor does not apply and must not be applied. It is a booking enquiry containing a name and a phone number.

Proposed: **90 days from creation**, purged by a scheduled job, configurable. It is long enough to cover a booking made well ahead and a patient who reschedules twice, and short enough that the clinic is not holding a list of strangers' phone numbers indefinitely under UU PDP 27/2022.

A prospective record that has been **LINKED** or **CONVERTED** is kept — it is the provenance of a real patient's first contact, and it is what makes a repeat booking from the same number resolve to the same person.

What this does *not* change

- **The chatbot's data limits stay.** Name and phone number remain everything it collects. This document does not propose asking a stranger for a date of birth over WhatsApp.
- **PCS-T08 's worklist and merge dialog stay.** They gain a second source — prospective records rather than draft profiles — and lose nothing.
- **Completion still happens through the patient-edit route**, for the reason already recorded: one write path for encrypted identifiers.
- **The MRN allocator is untouched.** It is simply called later, and less often.

Task outline

ID	TASK	EST
P17-T01	ProspectivePatient model, status enum, repository, retention column	5
P17-T02	Appointment dual FK + CHECK; scheduling and queue reads updated to resolve either side	8
P17-T03	Chat booking writes a prospective record instead of a draft profile (PCS-T07 path)	5
P17-T04	Arrival conversion: search-first, link-to-existing, convert-to-new with MRN allocation, appointment repoint — extending the shipped PCS-T08 surfaces	8
P17-T05	Backfill migration with a dry-run report, then the NOT NULL tightening as a separate release	8
P17-T06	Expiry job for unconverted prospective records	3

≈ 37 points. P17-T05 is the one with real risk and should not be scheduled in the same sprint as P17-T02 .

Open questions

#	QUESTION	OWNER	BLOCKING?
Q1	Is 90 days the right expiry for an unconverted prospective record?	Product + records officer	No
Q2	How many CHANNEL_BOOKING profiles exist in production today, and how many have clinical activity? The release-2 plan is shaped by the answer, and it should be measured before it is designed further.	Engineering	Yes — blocks P17-T05
Q3	Should a prospective patient be visible in the ordinary patient search, clearly badged, or only in the arrival worklist? Visible risks a clerk treating one as a patient; hidden risks a duplicate.	Clinical lead + front desk	Yes — blocks P17-T04
Q4	If someone books three times and never arrives, is that one prospective record or three? Matching on a normalised phone number would collapse them.	Product	No

Markdown source: docs/product/first-timer-registration-flow.md . Companion to the Phase 16 PRD, which raised this as RQ-3.